

FREE GUIDE • 2026

The AI Coding Playbook.

How to run AI coding agents like a team, not a genie. The process I use every day, in plain words.

Dmytro Shved

Independent AI Engineer · 17 years in software

What's inside

AI coding agents are fast. Left alone, they are also fast at building the wrong thing, guessing at bugs and saying "done" when it isn't. This playbook is the set of habits that fixes that.

01 The one idea

02 Where the rules live

03 Match process to task

04 Step 0: the brief

05 Look before you build

06 The task contract

07 Never accept "done"

08 Picking the model

09 Root cause, not loops

10 A bug is a shape

11 Testing

12 AI review must prove it

13 Memory and decisions

14 Hooks and gates

15 What to type

16 Red flags

Works with Claude Code and any agent that can run sub-agents, read rule files and use skills. The ideas work even if you only use one chat window.

01

The one idea

The main model is the lead. Agents are hands with no memory.

The session you talk to holds the context: what you asked for, the design, the decisions made an hour ago. It plans, decides, writes the tasks and checks the results.

Cheaper agents do the volume: searching, mechanical building, running tests, git.

Everything in this playbook serves three goals:

- 1 Understand** the task before building.
- 2 Fix at the cause**, not where the problem shows.
- 3 Check before you ship**, not after.

Rule of thumb: delegate when the task is explicit. Keep it when the knowledge is in your head.

Where the rules live

Rules come in layers that load at different moments. The always-on file stays short.

GLOBAL RULES

Every project. How the AI talks to you, which model does what, your quality rules.

PROJECT MAP

Every session in this repo. Standing rules, test policy, recent decisions. **Keep it short.**

PATH RULES

Load by themselves when a matching file is opened. Detail lives here.

SKILLS

Step-by-step workflows the agent loads only when the job comes up.

AGENTS

Helpers with a fixed role, model and effort level.

HOOKS

Small scripts that run at fixed moments.
"Remember to" becomes "can't forget".

MEMORY

Lessons and hand-off notes carried between sessions.

Watch out: a rules file that keeps growing stops being followed. Split it.

Match the process to the task

TINY CHANGE

One line, copy, config, rename. Just do it. No brief, no agents.

QUESTION

Answer it.

WIDE SEARCH

A cheap search agent. No contract needed.

BUG, ANY SIZE

Bug brief → find the root cause → sweep for twins. Never skipped, even for one line.

FEATURE

Brief → contracts for agents → verify → audit before commit.

NOT SURE

Ask one line: "full flow, or quick fix?"

Don't over-process small work. A contract longer than the change has failed. Three agents for two files is too many. If you already know what to type, type it.

Step 0: the brief

The brief adds no new rules. It's a moment where work stops and runs the rules you already have. Before the first edit:

- 1 **Pin the ask** in one sentence, then its narrowest reading. Sort every wish by one test: does the core job need it?
- 2 **Read what's binding:** the design, the spec, past decisions. Was this tried and dropped before?
- 3 **Find what exists.** "Already does this: X" or "nothing found, I searched Y".
- 4 **Walk the flow for gaps:** empty state, errors, permissions, the other platform, emails and exports, old data.
- 5 **Name where it goes in:** the one place that owns the behavior.
- 6 **Define "done"** as checks a person can see.

Ask the human only real product questions, each with a recommendation. Technical choices: decide, note why, move on.

The bug brief and the loop alarm

Four questions before fixing any bug:

- 1 **Has this happened before?** The same symptom coming back means the last fix was a patch.
- 2 **Seen, not guessed.** A log line, a real record. No evidence, no fix.
- 3 **How many places make this decision?** Two places deciding differently = nobody owns it.
- 4 **Blast radius.** Who else calls the code you're about to change?

Run a quick check mid-task when:

- someone says "still broken", "again", "not fixed"
- you're about to make a second fix in the same area
- the work reached a file the brief didn't name
- you're about to say "done"

Look before you build

Before anything bigger than a one-line change, say in one line what already does this. Search for the **behavior**, not the name. Duplicates rarely have the same name.

EXACT MATCH

Use it. A second copy is a bug waiting to happen.

DOES MORE

Reuse it, or split the shared core out. Never copy the half you need.

NEARLY RIGHT

Badly named or placed? Fix it in place. Don't build a sibling.

ALREADY DONE

Or not needed? Say so and stop. That's a good result.

Why: AI agents love writing new code. Two copies of the same rule drift apart, and the next bug gets fixed in only one of them.

The task contract

An agent starts blank. It can't see your conversation or the decision you made an hour ago. **If it isn't in the task, it doesn't exist.**

GOAL	one sentence: what's true when done
FILES	exact files to create or edit
SIZE	expected size, e.g. ~20 lines, 2 files
MIRROR	the existing file to copy the style from
BINDING	design, spec, rules it must read
DECISIONS	choices made, incl. what was REJECTED
INTERFACE	exact API shape, if others depend on it
INVARIANTS	what must still be true after
DO NOT TOUCH	files to leave alone, and why
DONE WHEN	checks a person can see
VERIFY	exact test command; new tests red first
IF UNSURE	stop and ask, with a proposed answer
RETURN	changed files, real test output

A tight task is cheaper **and** more accurate. A vague one makes the agent explore and invent.

No extras

Overbuilding is the quiet failure. Paste this into every task:

Build the smallest change that meets GOAL and DONE WHEN. No new abstraction, wrapper or helper unless named. No config flags. No drive-by refactors. No defensive code for situations not in this task. If the only way forward is a big new structure, the task is wrong: stop and ask.

What the lead never hands off:

- product and architecture decisions
- edits that depend on the conversation or a subtle rule: permissions, state changes, data migrations
- fix-ups after a review
- final integration and checking
- writing the task itself

Never accept “done”

An agent saying "done" is a claim, not proof. For every returned task:

- 1 Read the real changes, only the files it was given.
- 2 Check each "must still be true" rule, one by one.
- 3 **Check what it removed.** This is where agents fail most.
- 4 **Check what it added that nobody asked for.** Take it out now.
- 5 Check it stayed inside its files.
- 6 Check your own earlier edits weren't overwritten.
- 7 Check the output it pasted is real, not a summary.
- 8 Re-run the tests yourself for anything that could ship wrong.

Never correct a running agent. A message mid-build gives you two half-plans. Let it finish or stop it, fix the task, run it fresh.

Picking the model

Always set the model on every agent call. In many tools, leaving it out quietly uses the most expensive one.

CHEAPEST

Run tests, run git. Report only, never fix.

CHEAPER

Search, research, mechanical building.

STRONGEST

Root-cause analysis, reviews, risky building.

One question decides: "If this came back slightly wrong, would my tests catch it?" Yes → cheaper model. No → strongest.

"No" is a short, closed list: who someone is (login, permissions), money, anything that sends, data migrations, status changes, safety, a brand-new rule.

"It feels important" is not on the list. And a cheaper model never gets a thinner task. A thin task is exactly what makes a cheap model invent.

Running agents in parallel

- **Fix the interface first.** When backend, web and mobile share an API, write the exact shape before any agent starts and paste it into every task. Three agents left alone invent three different APIs.
- **Only tasks that touch different files run at the same time.** Same file = run them one after another.
- **Three or more tasks?** Save them as files, not just chat. Decisions that live only in chat get lost when the conversation is compressed.
- **30-second check before starting:** no file in two tasks, the interface is in every task, every referenced file exists.
- **Integrate yourself.** You hold the rules no single agent could see.

Shared folder? Ban "undo everything" commands in agent tasks. One careless undo can wipe everyone's unsaved work.

Root cause, not loops

Fix where the problem **starts**, not where it shows. A patch makes one case pass and leaves the cause alive. Then the patches start fighting each other.

Before the first edit: what I see → why (keep asking until the answer is a decision, not a line of code) → the one place that owns it.

Warning signs of a patch:

- a new "if" for this one case, word or user
- a word added to a list, a magic value, a delay, a swallowed error
- the same correction added in a second place
- a test changed only so it passes

PATCH

The bot misread "no" once, so add that phrase to a word list.

ROOT FIX

Fix the one part that decides "what did the user mean", and test it on many real conversations.

A bug is a shape, not a line

The place a bug was reported is where it was **found**, not where it **is**. Search five directions for its twins:

SAME MISUSE

Every other call made the same wrong way.

SAME DATA

Every place this field is read or written.

SAME RULE

Every place that re-decides it instead of asking one owner.

OTHER PLATFORM

Web, mobile, email, PDF, exports.

OTHER DOOR

Background jobs, AI tools, admin paths, scripts.
Missed most often.

Then close it so it can't come back: one chokepoint, a database rule, a test you've seen fail, or a lint rule.

"Found 0 others" is a claim about code that **runs**. A five-minute test beats an hour of reading.

Testing

- **Reproduce first.** Write the cause in one sentence with the evidence next to it.
- **Red before green.** Watch every new test fail on the old code. A test you never saw fail proves nothing.
- **One status, one writer.** One function changes a status, and it checks what's allowed.
- **Run it, don't just read it.** Green types are not proof. Use the real screen or endpoint once.
- **Test only what you touched.** Batch your edits, then test once.

Your test data must try to break your fix:

- a value stored the old way
- a field that really is empty
- two where the code expects one, none where it expects some
- an old record a filter should skip

Can't name data that would break your guard? Then you don't understand the guard yet.

AI review must prove it

AI reviewers write convincing text. Many findings sound right and are wrong, and each one costs you time.

A finding is a demonstration, not an argument.

- 1 Scouts** read the change and list leads only. "No leads" is a valid answer.
- 2 Investigators** take one lead each, in a throwaway copy of the code. Default verdict: not real.
- 3** A lead becomes a finding only with a **failing test that was run**, plus the passing case to prove the test works.
- 4** The audit **fixes nothing**. It reports. You fix one at a time, with a test in between.

Audit **before** you push. Not after. And scale it to the change: a copy edit needs nothing, money or logins always get the full check.

Memory and decisions

The AI forgets everything when a session ends. Anything not written down is lost.

- **Decisions log.** Every real product or design decision, newest first. Check it before proposing a change: an idea already tried and dropped is ping-pong, not progress.
- **"START HERE" notes.** At the end of a session: where things stand, what's pushed, what's waiting.
- **Feedback memories.** When you correct how the AI works, save it with the reason. Never correct the same thing twice.
- **Don't save** what the code or history already says.
- **A memory is a snapshot.** Check the file it mentions still exists.

Bonus: let the AI write your daily and weekly reports from the git history, in plain words.

Hooks and gates

A written rule can be forgotten. A check that fails can't.

Hooks are small scripts that run at fixed moments:

- **Loop alarm:** when your message says "still", "again" or "not fixed", the AI is reminded to stop and find the real cause before touching code.
- **Before tools run:** block writing outside the project, block badly formed commits.
- **Long session warning:** before compression loses detail.

Gates fail the build: a test that reads your rules list and fails if a rule loses its check, a slow-test gate, a "no hardcoded colors" gate, a time-zone gate.

Long-term goal: every rule you keep repeating becomes a check that goes red.

What to type

WEAK BUG REPORT

"The supplier page is broken, fix it."

STRONG BUG REPORT

"On the test site, order #1234: Reply sent the email to our own address, not the supplier. Today 14:05. Second time, it was "fixed" last week."

Give real evidence: which record, what time, what you expected. And say if it happened before. That one fact changes how the AI should approach it.

Make the AI talk back like a human:

- the topic and the answer in the first line
- about three lines, plain words
- questions in their own block at the end, with options and a recommendation
- never "done" when part is unfinished or untested

Red flags: stop if you see one

- Code written before anyone read the design or past decisions.
- A fix with no reproduction ("it's probably...").
- The same symptom fixed twice with no pause in between.
- A new "if", list word, flag or delay for one case.
- A test never seen failing, or edited just to pass.
- An agent started with no model, or a task without a test command.
- Accepting an agent's summary without reading the changes.
- A correction sent to a running agent.
- An abstraction or flag nobody asked for.
- An AI review "finding" with no failing test.
- "Web works" reported as "the feature is done".
- A decision made in chat that never got written down.

About the author

Dmytro Shved, Independent AI Engineer. 17 years in software engineering across front end, back end and mobile, including lead roles and VP of Engineering.

I build AI agents and LLM features for production, and I set up this way of working for engineering teams.

If you want it in yours, send me a message on LinkedIn.

If this guide was useful, pass it on to one developer who would use it.